Appendix A

RexBERT: Turning a Domain-Specialized Encoder into a Sentence-Embedding Model

After reading this appendix, you will be able to:

- Explain why the pooled hidden states of a masked language model are a weak similarity space, and name two diagnostics that reveal the problem.
- Convert a Hugging Face encoder into a sentence encoder by adding pooling and normalization and fine-tuning with in-batch negatives, and choose among multiple-negatives ranking, CoSENT, and TSDAE from the supervision on hand.
- Build contrastive pairs from session logs without placing evaluation sessions in the training pairs, and select checkpoints with a retrieval metric on held-out sessions.
- Design a comparison that isolates one factor, here the domain phase of pretraining, by holding architecture, tokenizer, data, and recipe fixed, and report it with seeds and confidence intervals.
- Decide whether a domain-specialized encoder is worth adopting, given the supervision, vocabulary, language, text length, and serving cost of a catalog.

A.1 The Question Chapter 10 Left Open

A.1.1 A model that knows the words and still needs a similarity space

A catalog encoder has two jobs. It reads product language: titles, brands, sizes, and short descriptions. It also places related products near one another, so a nearest-neighbor lookup returns substitutes. A masked language model is trained for the first job. Given a text with some tokens hidden, it predicts those tokens from the rest (Devlin et al., 2019). The loss never compares two complete inputs.

Cosine similarity is the inner product of two vectors after each is scaled to length 1. A useful embedding space gives substitutes a higher cosine than a random pair of catalog items. Masked-token training does not set that requirement.

RexBERT is a family of masked language models whose last pretraining stage uses e-commerce text (Bajaj and Garg, 2026). This appendix calls that released checkpoint the released RexBERT encoder. When the authors evaluate semantic similarity on the Shopping Queries dataset, ESCI, they first map the four relevance grades to scores and fine-tune with CoSENT, a loss that ranks pairs by cosine (Huang et al., 2024; Reddy et al., 2022). They do not publish a similarity number from the raw pooled states. Reimers and Gurevych (2019) had already found that averaged BERT token vectors scored below averaged GloVe vectors on semantic textual similarity benchmarks. Both groups treat a pretrained encoder as a backbone that still needs a similarity objective.

The model this appendix trains is a different object: the converted RexBERT sentence encoder. It keeps the backbone, adds mean pooling and L2 normalization, and fine-tunes on behavioral pairs. Chapter 2 points to RexBERT for e-commerce embeddings (Section 2.4.8). Chapter 10 describes the released

encoder, trained to predict masked tokens, with no contrastive objective. The two descriptions match the two objects. The names have to stay apart.

A.1.2 What Chapter 10 compared, and what it left open

Section 10.4 compares general-purpose sentence encoders, already trained with a contrastive objective, with RexBERT used zero-shot. Zero-shot here means the pooled states of the released encoder, with no session fine-tuning. On Amazon KDD retrieval, the general-purpose encoders ranked above zero-shot RexBERT. Section 10.4 reads that gap as evidence that the training objective matters more than domain coverage for embedding quality, and it says further datasets would strengthen the conclusion. Section 10.6 then fine-tunes a sentence encoder that already carries a contrastive objective, using multiple-negatives ranking on behavioral pairs. Chapter 10 does not put RexBERT and a general-domain sibling through that same objective and compare them.

Those are different claims. The first is about zero-shot geometry: an encoder trained only to predict tokens is a weak similarity space, including when its text is in domain. The second is about residual value: once every candidate has the same contrastive recipe on the same sessions, does the e-commerce phase still move retrieval? Only the first claim was tested in Chapter 10. This appendix repeats the zero-shot comparison under a written protocol, then asks the second question.

A.1.3 Roadmap

Section A.2 describes RexBERT and the sibling that makes the comparison possible. Section A.3 explains the weak geometry and shows the cosines measured on this catalog. Section A.4 is the conversion recipe, for any encoder from the Hugging Face hub, the public store of model weights this book uses for text encoders. Section A.5 reports the UK session experiment. Section A.6 is the decision guide. Section A.7 states the rule this protocol supports.

A.2 What RexBERT Is

A.2.1 Architecture

RexBERT is a ModernBERT encoder (Warner et al., 2024; Bajaj and Garg, 2026). Rotary position embeddings replace a learned vector per absolute index, which is how the family trains a context of 8,192 tokens. The feed-forward block uses GeGLU, a gated linear unit. Attention alternates between a full-sequence layer and a local window, and unpadding drops pad tokens from that computation. None of these choices trains a similarity space. They change how the encoder reads a long sequence. The similarity objective is Section A.4.

The authors report four sizes, about 17 million, 68 million, 150 million, and 400 million parameters: micro, mini, base, and large (Bajaj and Garg, 2026). Table A.1 lists the shapes. This appendix fine-tunes micro. RexBERT-base appears only in the zero-shot replication, so the controlled comparison is at the small end of the family.

A.2.2 Where RexBERT and Ettin share a trunk

The curriculum has three phases (Bajaj and Garg, 2026). Phase 1 is about 1.7 trillion tokens of mixed text at a short context, stated as 512 tokens. Phase 2 spends about 250 billion tokens extending the context to 8,192. Phase 3 anneals on e-commerce text for about 350 billion tokens, with Guided MLM, a mask that prefers entities and attributes, and a lower masking rate. The budgets sum to about 2.3 trillion

tokens, of which the e-commerce portion is the last 350 billion. The paper's acknowledgement is the sentence the comparison uses: phases 1 and 2 mirror the Ettin recipe, and annealing starts from Ettin's 8,192-token checkpoint (Bajaj and Garg, 2026; Weller et al., 2025).

Ettin is an open suite of encoders and decoders trained with one recipe across sizes (Weller et al., 2025). The 17-million-parameter encoder, `ettin-encoder-17m`, is the control. Weller et al. (2025) report a 50 billion token decay. The released Ettin-17m model card reports 100 billion tokens of higher-quality general text, with the learning rate decayed. Figure A.1 draws the branch. The two 17-million-parameter checkpoints share the ModernBERT masked-language-model class, the shape in Table A.1, and a byte-pair tokenizer derived from OLMo with 50,368 tokens (Bajaj and Garg, 2026). Hub tags checked on 13 November 2026 mark RexBERT-micro Apache-2.0 and Ettin-17m MIT. The revisions pinned in the companion code match those hub commits.

The third phase differs in two ways at once. The mixture differs: e-commerce annealing against a general-domain decay. The length differs under either budget: about 350 billion tokens of e-commerce text against the paper's 50 billion token decay or the released model card's 100 billion. A RexBERT gain is evidence about that phase as released, mixture and length together, under either budget. It is not a measurement that holds token count fixed. Section A.5 repeats this confound next to the number.

A.2.3 The corpus, and what domain pretraining has been shown to do

Ecom-niverse is the authors' filter of FineFineWeb: English pages from commerce-leaning domains, a language-model label on a sample, a fastText classifier distilled from those labels, and a threshold (Bajaj and Garg, 2026). That is their account. This appendix does not re-derive the filter.

Continued pretraining on in-domain text, then task fine-tuning, improved classification in four domains (Gururangan et al., 2020). BioBERT and SciBERT are the familiar cases (Lee et al., 2020; Beltagy et al., 2019). SciBERT also built a new vocabulary. RexBERT kept the general tokenizer, so odd brand strings and product codes are split by a vocabulary that was not fit to catalogs. Section A.6 returns to that limit.

Classification with a fine-tuned head is a different question from cosine retrieval of the next item. Domain-adaptive pretraining can be the right investment for token classification and still add little once a contrastive loss has been applied to both a domain encoder and its general sibling. The RexBERT authors' token-recovery and GLUE numbers are task fine-tunes, and the similarity number they publish is CoSENT on ESCI grades (Bajaj and Garg, 2026). Section A.5 measures the embedding question at micro scale.

Table A.1: RexBERT, the Ettin encoders at the same widths, and all-MiniLM-L6-v2. RexBERT shapes are the authors' configuration table. Ettin at 17M, 68M, and 150M matches it. MiniLM truncates at 256 word pieces by default (Sentence-Transformers, 2021). ModernBERT-family context is the 8,192-token length the papers train toward.

Licenses were checked on the hub on 13 November 2026 for RexBERT-micro and Ettin-17m. Confirm the card before adopting any other row.

Model	Parameters	Layers	Hidden	Heads	Output dim.	Context	Final phase	License
RexBERT-micro	17M	7	256	4	256	8,192	E-commerce anneal, about 350B tokens	Apache-2.0
RexBERT-mini	68M	19	512	8	512	8,192	Same curriculum as micro	Apache-2.0
RexBERT-base	150M	22	768	12	768	8,192	Same curriculum as micro	Apache-2.0
RexBERT-large	400M	28	1,024	16	1,024	8,192	Same curriculum as micro	Apac

Model	Parameters	Layers	Hidden	Heads	Output dim.	Context	Final phase	License
								he-2.0
Ettin-17m	17M	7	256	4	256	8,192	General decay, paper 50B; model card 100B	MIT
Ettin-68m	68M	19	512	8	512	8,192	General decay	MIT
Ettin-150m	150M	22	768	12	768	8,192	General decay	MIT
all-MiniLM-L6-v2	23M	6	384	12	384	256 word pieces	Contrastive training on over 1B pairs	Apache-2.0

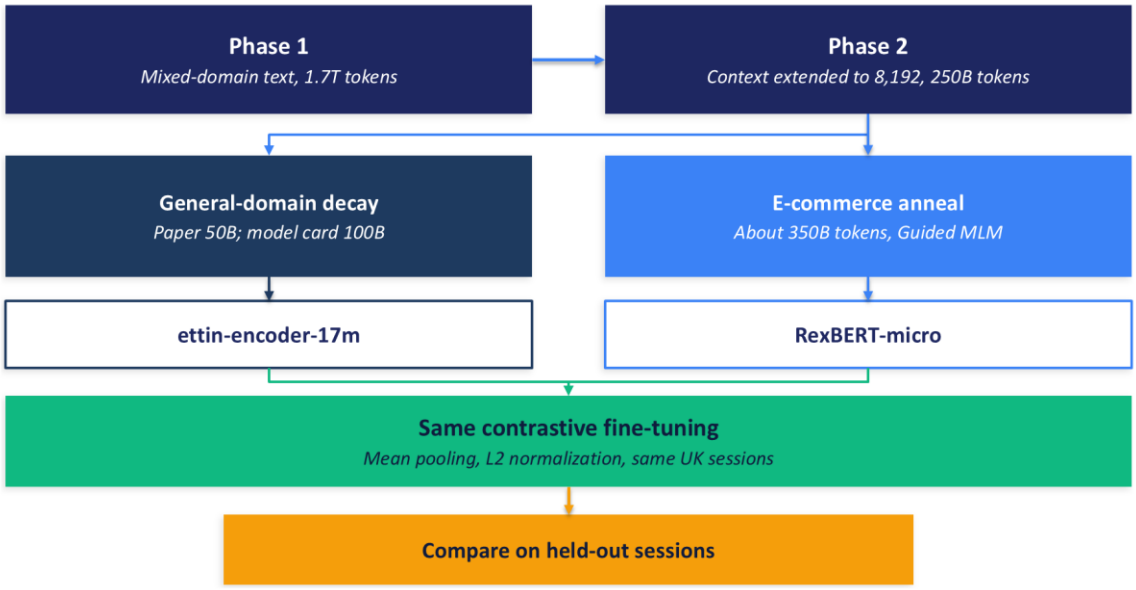


Figure A.1: RexBERT-micro and Ettin-17m share the early curriculum and split at the third phase. Both then enter one contrastive recipe on the same sessions.

As of 13 November 2026 the checkpoints used here are public. The same account also hosts experimental similarity models and rerankers, separate from this backbone. Pin a revision. The companion code already pins the five it loads.

The code includes a fill-mask demonstration for Exercise A.1. Token predictions are not a retrieval result, and this appendix does not treat them as one. Section 5.3.4 is the book's other use of a masked language model in retrieval: SPLADE reads the MLM head's vocabulary logits. Figure A.2 is the conversion Sections A.3 and A.4 explain. The head is set aside, pooling and normalization make the vector, and a batch of session pairs supplies the loss.

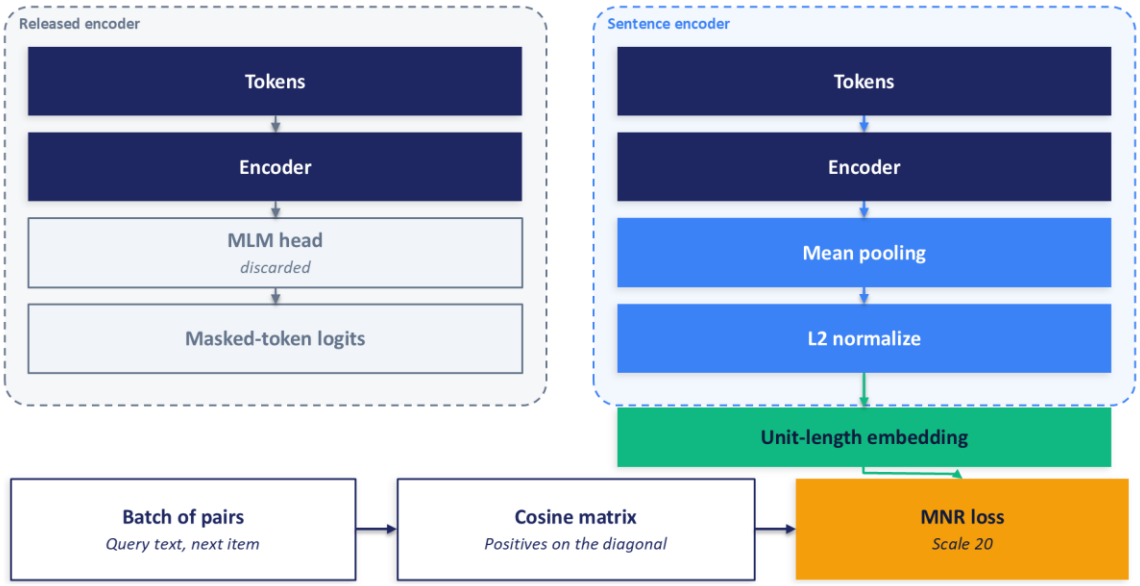


Figure A.2: The released encoder ends in a masked-token head that the sentence encoder discards. Mean pooling and L2 normalization produce the vector, and multiple-negatives ranking trains it on session pairs. Section 10.6 derives the loss.

A.3 Why a Language Model Is Not Yet an Embedding Model

A.3.1 What the masked-language-model objective rewards

The masked language model scores a token in context. The loss is the log probability of each hidden token. A step can succeed while two substitutes sit far apart. The objective has no term for that. Reimers and Gurevych (2019) found that pooled BERT states, with no similarity fine-tuning, lagged averaged GloVe vectors on similarity benchmarks. Section 10.4 uses that history as the reason sentence-BERT exists. This section adds the geometry, then the measurement on this catalog.

A.3.2 Anisotropy

Contextual word vectors from BERT-style models occupy a narrow cone in the upper layers. Ethayarajh (2019) showed that randomly chosen words then have a high cosine, so the representation is anisotropic: the directions that are used in practice cover a small part of the space. At sentence scale, Li et al. (2020) found the same cone, and a further skew from token frequency, so common tokens tug the pooled vector. Cosine, which assumes that angle tracks meaning, becomes a weak ranker when almost every pair has a similar angle.

A contrastive objective pushes back on both problems. Gao et al. (2021) describe the effect as alignment plus uniformity. Alignment pulls a positive pair together. Uniformity spreads unrelated points so they do not all collapse into the cone. Multiple-negatives ranking, the loss in Section A.4 and in Section 10.6, is a contrastive loss of that family. The zero-shot RexBERT encoder has had the masked-token loss and has not had this one.

A.3.3 Two retrieval symptoms, and which one this protocol measured

The geometry shows up in retrieval in two ways that are worth separating, because only one of them was measured here.

The first symptom is that neighbors look uniformly similar. If random catalog pairs already have cosine near 1, the rank order is a contest among tiny gaps. A small change in pooling, length, or a frequent token can reorder the list. The second symptom is that neighbors follow surface tokens. The nearest items share a brand, a size, or a color word, and they do not share a function a shopper would substitute. The second symptom is a diagnostic a team can run by reading neighbors. This protocol did not score token overlap of neighbors, so it does not report that symptom as a finding.

The first symptom is in Figure A.3. The figure is two densities, each from 10,000 pairs, for three encoders: RexBERT-micro with no contrastive fine-tuning, the same model after fine-tuning at seed 0, and all-MiniLM-L6-v2 with no further fine-tuning on these sessions. On random UK catalog pairs, zero-shot RexBERT-micro has mean cosine 0.98, and 9,999 of the 10,000 pairs sit above 0.8, and the lowest is 0.69. On held-out next-item pairs the mean is 0.99. The gap between a random pair and a behavioral pair is about one hundredth. After fine-tuning, random pairs fall to mean cosine 0.39 and next-item pairs sit at mean 0.79, so the positive pairs pull away from the background. MiniLM, which was contrastively pretrained before this experiment, already separates them at zero-shot: mean 0.12 on random pairs and 0.61 on next-item pairs.

The same diagnostic file records Ettin-17m at zero-shot, which Figure A.3 does not draw. Its random-pair mean cosine is 0.99. The cone is a property of these masked-language-model checkpoints on this catalog, not a quirk of the e-commerce anneal. Fine-tuning is what opens the space. Section A.5 asks whether, once both siblings have been opened with the same recipe, the anneal still changes retrieval.

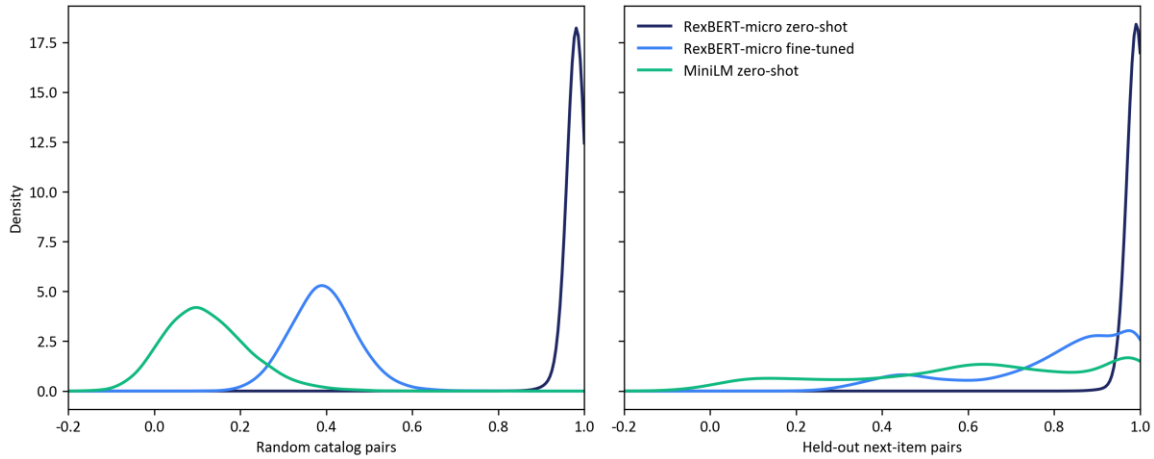


Figure A.3: Cosine similarity for 10,000 random UK catalog pairs and 10,000 held-out next-item pairs. Zero-shot RexBERT-micro concentrates near 1 on both sets. Fine-tuning spreads random pairs and leaves next-item pairs higher. Zero-shot MiniLM is already spread.

A.3.4 Two ways to use a masked language model for retrieval

Section 5.3.4 uses the masked language model in a different way. SPLADE keeps the token-prediction head and reads its vocabulary logits as weights on terms, which produces a sparse vector a lexical index can store. The head is useful there because it was trained to say which words fit a context. A pooled hidden state is another object. It is a single dense vector, and nothing in masked-token training

calibrated its cosine. The conversion in Section A.4 discards the head, pools the states, and trains the cosine. Both uses can sit in one retrieval stack. They answer different questions, and Appendix C is where this book combines dense and sparse retrieval.

A.4 Converting an Encoder into a Sentence Encoder

The recipe in this section is the one the experiment runs, and it is written so the same steps apply to any encoder the Hugging Face libraries can load. Sentence-transformers is the Python library that stacks those steps: a transformer module, a pooling module, a normalization module, and a trainer. The companion code lives in `code/appendix_rexbert/`. The entry point is `run_all.py`, with two profiles, `cpu_small` and `gpu_full`. Reported numbers in Section A.5 come from `gpu_full`. The `cpu_small` profile walks the same code on a smaller sample, and its metrics are a teaching aid, not the results below. `PREREGISTRATION.md` records the questions and the decision rule. Measured runtimes for both profiles are in the companion README.

A.4.1 Pooling, normalization, and the discarded head

The converted model is three modules. The transformer is the released encoder, loaded at a pinned revision with remote code disabled. Mean pooling averages the token states, using the attention mask so pad positions do not enter the average. L2 normalization scales the result to length 1, which makes the inner product equal cosine. Reimers and Gurevych (2019) compared pooling choices for their sentence encoder and found mean pooling the strongest in that ablation, which is why it is the default here. The masked-token head is not part of the stack. Listing A.1 is the construction, from the companion file `encoders.py`.

```
        transformer = Transformer(
            spec.hub_id,
            max_seq_length=max_seq_length,
            model_kwargs={"revision": spec.revision, "trust_remote_code": False},
            processor_kwargs={"revision": spec.revision, "trust_remote_code": False},
            config_kwargs={"revision": spec.revision, "trust_remote_code": False},
        )
        auto_map = getattr(getattr(transformer, "model", None), "config", None)
        auto_map = getattr(auto_map, "auto_map", None)
        if auto_map:
            raise RuntimeError(f"{spec.hub_id} requested remote code")
        dim = int(transformer.get_embedding_dimension())
        if dim != spec.expected_dim:
            raise ValueError(f"{spec.key} embedding dim {dim} !=
{spec.expected_dim}")
        model = SentenceTransformer(
            modules=[transformer, Pooling(dim, pooling_mode="mean"), Normalize()],
            device=device,
            trust_remote_code=False,
            similarity_fn_name="cosine",
        )
```

Listing A.1: Building the sentence encoder from a masked language model in `encoders.py`. Mean pooling and L2 normalization follow the transformer. The masked-token head is omitted.

An encoder that is already a sentence-transformer model, such as all-MiniLM-L6-v2, is loaded as that model and then fine-tuned under the same loss, length, and batch. The construction above is for a masked language model that has no pooling module of its own. Figure A.2 shows the same stack: the discarded head on the left, pooling and normalization on the right, and the batch loss along the bottom.

A.4.2 Choosing the objective from the supervision on hand

Table A.2 matches the objective to the supervision. Section 10.6 derives multiple-negatives ranking. The only addition here is the scale. The loss multiplies cosine by a scale s before the softmax. Temperature is the reciprocal, $\tau = 1/s$. This recipe sets $s = 20$, so $\tau = 0.05$.

Multiple-negatives ranking applies when positive pairs exist and grades do not (Henderson et al., 2017; van den Oord et al., 2018). Each row of the batch is a query text and a positive text, and every other positive is a negative. A batch of 128 pairs supplies 127 negatives per query, which is why Section A.5 holds the batch size fixed. Section 5.4.3 makes the same point for two-tower models: the negatives define the contrast. CoSENT applies when labels are graded, as with the ESCI grades, and it is the loss the RexBERT authors used for similarity (Huang et al., 2024; Bajaj and Garg, 2026). The main experiment has session pairs, so it uses multiple-negatives ranking. Exercise A.3 is the graded comparison. TSDAE applies when the only domain text is unlabeled: the model reconstructs a corrupted sentence, and the encoder is kept (Wang et al., 2021). It is the fallback in Section A.6. Triplet loss uses one negative per anchor and spends the batch on fewer comparisons. It fits when negatives are mined and the batch must stay small.

Table A.2: Objectives for turning an encoder into a sentence encoder. The experiment in Section A.5 uses multiple-negatives ranking because the supervision is a positive pair from a session.

Objective	Supervision	What training changes	When to use it
Multiple-negatives ranking	Positive pairs	Cosine of the true pair against the other items in the batch	Sessions, co-clicks, or query-click logs
CoSENT	Graded similarity	Order of pairwise cosines against the grades	ESCI-style labels
TSDAE	Unlabeled domain text	A denoising reconstruction; the encoder is kept	Catalog text and no behavioral pairs
Triplet loss	Anchor, positive, one negative	A margin against a single negative	Small batches or mined negatives

A.4.3 Pairs from sessions, without leaking the evaluation sessions

Amazon-M2 sessions are chronological product lists (Jin et al., 2023). The training table has previous items, a next item, and a locale, and it has no timestamp, so the protocol splits sessions rather than cutting by time. A hash of the row index with seed 42 assigns eight tenths of UK sessions to training. A training pair is the last previous item and the next item, taken only from a training session. Exact duplicates are dropped, and the list is capped at 500,000 pairs. Section A.5 reports the counts that cap produced.

A session whose next item equals the query is degenerate, because the target would be the query itself. A popular item that appears twice in one batch would be a negative against itself. The duplicate-free batch sampler refuses that collision. Listing A.2 shows the argument. Frequent items still dominate the list, because they are the next item in more sessions. Co-visitation in Section A.5 makes those frequencies explicit. Exercise A.2 rebuilds the pairs from every consecutive step. The co-visitation counts already use those steps. The contrastive pairs, as shipped, do not.

A.4.4 A learning rate for a model that has never seen the loss

MiniLM has already been trained with a contrastive loss on more than a billion pairs (Sentence-Transformers, 2021). RexBERT-micro and Ettin-17m have not. A fair comparison gives every model the same sweep and then freezes the chosen rate. The candidates are 2×10^{-5}, 5×10^{-5}, and 1×10^{-4}, one epoch on 100,000 pairs at seed 0. The rate with the highest validation score is kept, and the main runs restart from the original checkpoints at that rate. Section A.5 records which rate won. Held fixed: batch size 128, maximum length 128, the template `{title} [SEP] {brand} [SEP] {description}`, a warmup of one tenth, weight decay 0.01, and seeds 0, 1, and 2. The GPU profile trains in bfloat16, for up to three epochs, and evaluates the best validation checkpoint.

A.4.5 Selecting the checkpoint on a retrieval metric

The validation score is mean reciprocal rank at cutoff 100, computed by an information-retrieval evaluator on 2,000 validation sessions. The corpus for that evaluator is 50,000 products that include every validation target. The query item is masked so a product cannot count as a neighbor of itself. The validation scorer reads a host float32 copy of each encoder output. This validation score is not the test metric in Section A.5. The validation corpus is a sample, and the cutoff ignores ranks beyond 100, so the absolute number sits higher than full-catalog mean reciprocal rank. It is used only to pick a checkpoint.

Mean cosine on positive pairs is the wrong selection signal. Figure A.3 shows why. The zero-shot masked language models already assign a cosine near 1 to positive pairs, and they assign nearly the same cosine to random pairs. A collapsed space can look excellent on that average. Retrieval on held-out sessions cannot. Test sessions are scored once, after the checkpoints are chosen. Listing A.2 is the head of the training-argument block in `train.py`. `SentenceTransformerTrainer` receives the model, these arguments, the pair dataset, the multiple-negatives loss at scale 20, and the validation evaluator.

```
    return SentenceTransformerTrainingArguments(
        output_dir=output_dir,
        num_train_epochs=epochs,
        per_device_train_batch_size=batch_size,
        learning_rate=learning_rate,
        # Transformers 5 stores a fractional warmup as warmup_steps, not
warmup_ratio.
        warmup_steps=WARMUP_RATIO,
        weight_decay=WEIGHT_DECAY,
        bf16=bf16,
        fp16=False,
        batch_sampler=BatchSamplers.NO_DUPLICATES,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="eval_mrr_at_100",
        greater_is_better=True,
```

```
        seed=seed,
        data_seed=seed,
        logging_steps=50,
        save_total_limit=1,
```

Listing A.2: Training arguments in `train.py`. The duplicate-free sampler keeps a repeated item from becoming its own negative. The kept checkpoint is the one with the highest validation MRR at 100.

A.5 A Controlled Experiment on Amazon-M2

The questions, the metrics, and the decision rule were fixed in `PREREGISTRATION.md` before the fine-tuning runs. A difference is supported only when the paired 95 percent interval of the seed-averaged per-query difference excludes zero and all three seeds agree in sign. The interval is a bootstrap of 1,000 resamples of the test queries. A single run is a weak comparison (Bouthillier et al., 2021). Mean reciprocal rank on the full catalog is the primary metric. Recall@50 is reported beside it and does not choose the branch. The full protocol ran on a single cloud GPU.

A.5.1 Data, split, and metrics

The catalog is the UK locale of Amazon-M2, released for KDD Cup 2023 (Jin et al., 2023). The files hold 500,180 UK products and 1,182,181 UK sessions. Sessions are chronological product lists. The table has previous items, a next item, and a locale, and it has no timestamp, so the split cannot follow time. A hash of the row index with seed 42 assigns 945,368 sessions to training, 118,434 to validation, and 118,379 to test. Training pairs come only from training sessions: 945,368 last-to-next pairs, 764,393 after exact duplicates are removed, then a seeded cap of 500,000. Degenerate sessions, where the next item equals the query, number zero.

Products use the Chapter 10 template `{title} [SEP] {brand} [SEP] {description}`. The median description is 24 words and the 90th percentile is 53. The median session has three previous items. Of the products, 39,258 have an empty description. Every encoder truncates at 128 tokens, so the 8,192-token context is not what this comparison exercises. Chapter 3 defines the metrics in general. Here, with one target, Recall@K is 1 when that target is in the first K results. Mean reciprocal rank is the reciprocal of the target's rank in the full list. NDCG@10 gives partial credit when the target falls inside the top 10, with more credit for a higher rank.

The reported table uses a seeded sample of 20,000 of the 118,379 test sessions. The corpus is all 500,180 UK products. Search is exact inner product in a flat index. Unit-length embeddings make that inner product equal cosine. The query item is removed from its own results. A separate path repeats Chapter 10's zero-shot protocol: 5,000 queries with seed 42, and every later item that followed the query anywhere in the data counted as a target, 2.91 targets per query on average. Those numbers are a different scale.

The learning-rate sweep from Section A.4 selected 5×10^{-5} for RexBERT-micro, 1×10^{-4} for Ettin-17m, and 2×10^{-5} for MiniLM, each by validation mean reciprocal rank at cutoff 100. Main runs start from the original checkpoints at those rates, for up to three epochs, and keep the best validation checkpoint. The validation corpus is 50,000 products, so that selection score is not the test number in the table.

A.5.2 Baselines that do not read text

Popularity ranks products by how often they are the next item in training. Co-visitation counts how often B follows A across every consecutive step in training sessions, and falls back to popularity when a query has no neighbor. Co-visitation uses no title, brand, or description. If it leads an encoder, the encoder has not beaten a count of shopping transitions.

A.5.3 Three questions

Under Chapter 10's protocol, all-MiniLM-L6-v2 and all-mpnet-base-v2 each reach mean reciprocal rank 0.317. RexBERT-base reaches 0.285, RexBERT-micro 0.265, and Ettin-17m 0.252. Recall@10 follows: 0.295, 0.292, 0.249, 0.221, and 0.207. The direction agrees with Section 10.4. General-purpose sentence encoders rank above zero-shot RexBERT. The session-level task, before fine-tuning, has the same order: MiniLM 0.205, mpnet 0.204, RexBERT-base 0.181, RexBERT-micro 0.166, Ettin-17m 0.158. RexBERT-base and mpnet were not fine-tuned. The controlled comparison uses the three models that were.

Contrastive fine-tuning moves RexBERT-micro from 0.166 to 0.203 mean reciprocal rank on the 20,000 queries. The paired difference is 0.037, the interval is 0.034 to 0.039, and all three seeds are positive. That contrast is supported. Recall@50 on the same rows moves from 0.330 to 0.508. The pre-registered test of this question was the mean-reciprocal-rank contrast. Figure A.3 is the geometry of the same change.

With the recipe held fixed, fine-tuned RexBERT-micro minus fine-tuned Ettin-17m is 0.0031 mean reciprocal rank. The interval is 0.0015 to 0.0047. The seeds are 0.0041, 0.0017, and 0.0036, all positive. The rule calls the difference supported and positive. Once the objective is shared, the e-commerce phase is associated with a higher mean reciprocal rank than the general-domain decay, on this task, at this size. The third-phase token counts still differ, as Section A.2 said, so the 0.0031 includes that longer budget. On Recall@50 the difference is 0.0004, the interval is -0.0024 to 0.0031, and the seeds disagree in sign. Recall@50 does not support a difference, and both siblings land at 0.508.

Fine-tuned MiniLM reaches mean reciprocal rank 0.209 and Recall@50 of 0.530, ahead of both siblings. MiniLM was contrastively pretrained on more than a billion pairs (Sentence-Transformers, 2021), and its dimension is 384 against 256. It is the practical reference. The controlled contrast is RexBERT-micro against Ettin-17m.

A.5.4 Where co-visitation still leads

Table A.3 and Figure A.4 keep the baselines. Co-visitation's mean reciprocal rank is 0.235, interval 0.230 to 0.240, above fine-tuned MiniLM at 0.209, RexBERT-micro at 0.203, and Ettin-17m at 0.199. It also leads NDCG@10, at 0.261 against 0.240, 0.231, and 0.227. Popularity is below 0.001 on every column.

The lead sits on pairs training already recorded. Of the 20,000 queries, 5,587, about 28 percent, have a query-and-next pair that occurs in some training session, even though that session was held out. On those pairs co-visitation reaches mean reciprocal rank 0.673 and Recall@50 of 0.997. On the other 14,413 pairs it falls to 0.065, while fine-tuned RexBERT-micro, Ettin-17m, and MiniLM sit at 0.109, 0.107, and 0.115. On the full set the encoders lead Recall@50: 0.530, 0.508, and 0.508 against co-visitation's 0.383. Rank one rewards the memorized transition. The first 50 reward the encoder when the count is missing.

A cold target never appears in training. There are 368 such queries, about 2 percent. Cold Recall@50 is zero for both baselines. Fine-tuned cold Recall@50 is 0.455 for RexBERT-micro (0.408 to 0.505), 0.425

for Ettin-17m (0.378 to 0.472), and 0.469 for MiniLM (0.421 to 0.517). The point estimates lean toward RexBERT-micro over Ettin-17m, and the intervals overlap. The pre-registered rule was not applied here. The cold slice does not by itself establish a domain effect.

Table A.3: Next-item retrieval on 20,000 held-out UK sessions against all 500,180 UK products. Fine-tuned rows average three seeds. Intervals are 95 percent bootstrap intervals over queries. Cold Recall@50 uses the 368 queries whose target never appears in training.

Model	Setting	Recall @10	Recall@50	MRR	NDCG @10	Cold Recall@50
Popularity	Baseline	0.000	0.000 (0.000 to 0.001)	0.000 (0.000 to 0.000)	0.000	0.000
Co-visitation	Baseline	0.350	0.383 (0.377 to 0.390)	0.235 (0.230 to 0.240)	0.261	0.000
MiniLM	Zero-shot	0.347	0.482 (0.475 to 0.488)	0.205 (0.200 to 0.210)	0.232	0.427 (0.375 to 0.481)
RexBERT-micro	Zero-shot	0.271	0.330 (0.324 to 0.337)	0.166 (0.161 to 0.170)	0.188	0.261 (0.217 to 0.304)
Ettin-17m	Zero-shot	0.258	0.304 (0.298 to 0.310)	0.158 (0.154 to 0.162)	0.180	0.223 (0.182 to 0.266)
RexBERT-micro	Fine-tuned	0.351	0.508 (0.502 to 0.515)	0.203 (0.198 to 0.207)	0.231	0.455 (0.408 to 0.505)
Ettin-17m	Fine-tuned	0.346	0.508 (0.501 to 0.515)	0.199 (0.195 to 0.204)	0.227	0.425 (0.378 to 0.472)
MiniLM	Fine-tuned	0.367	0.530 (0.523 to 0.537)	0.209 (0.204 to 0.214)	0.240	0.469 (0.421 to 0.517)

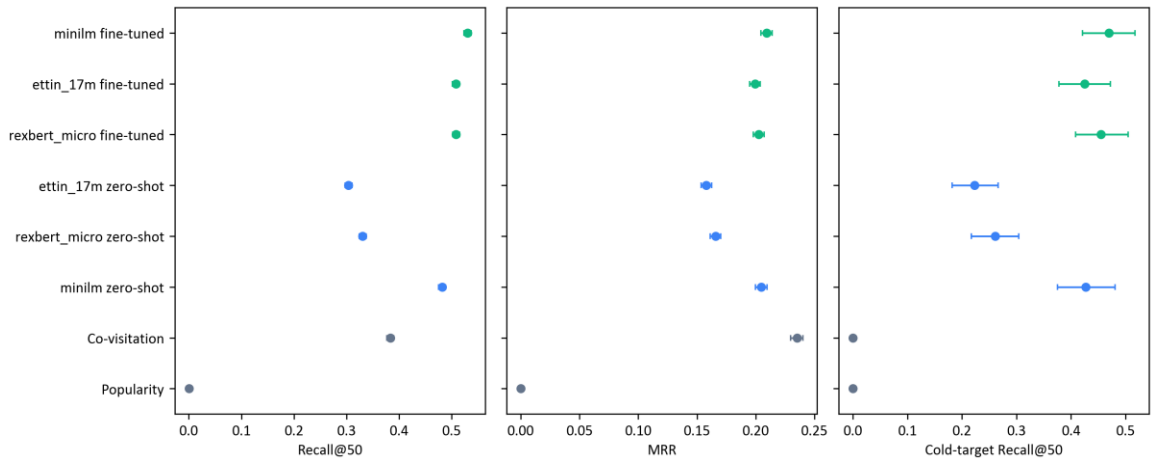


Figure A.4: Recall@50, mean reciprocal rank, and cold-target Recall@50 for Table A.3, with 95 percent intervals. Grey points are baselines, blue points are zero-shot encoders, and green points are seed-averaged fine-tunes. Row names are the companion code's short labels.

A.5.5 What one protocol can support

Contrastive fine-tuning raises RexBERT-micro by 0.037 mean reciprocal rank over its zero-shot encoder, and that gain is supported. After the same recipe, RexBERT-micro exceeds Ettin-17m by 0.0031, also supported, and Recall@50 does not separate them. Co-visitation remains the strongest mean-reciprocal-rank system on all 20,000 queries. The encoders lead Recall@50, cold targets, and pairs the counts have never seen. MiniLM leads both siblings after fine-tuning.

The scope is one locale, one pair definition, a split that is not temporal, and the micro size. It does not speak for RexBERT-base or RexBERT-large under this fine-tune, for other catalogs, or for ESCI search. On Recall@50 this table would not justify the domain phase over the general sibling. On mean reciprocal rank it records a small supported gain, still to be read beside the co-visitation count.

A.6 When Is a Domain-Specialized Encoder Worth It?

Section A.5 supports a limited reading of the domain phase. After one contrastive recipe, RexBERT-micro's mean reciprocal rank exceeds Ettin-17m's by a margin that passes the pre-registered rule, and Recall@50 does not. Each row below has a falsifier, a measurement that would retire the recommendation. Table A.4 collects them. The checks at the end catch an MLM encoder whose pooled states are placed straight into an index.

Supervision comes first. With catalog text only, start from a sentence encoder that has already had contrastive training. Zero-shot RexBERT-micro is the caution: mean reciprocal rank 0.166 against MiniLM's 0.205, and a random-pair cosine near 0.98. If the catalog's wording is far from that general encoder, TSDAE on the catalog text adapts the model without invented labels (Wang et al., 2021). Retire the advice if the domain encoder's zero-shot retrieval matches the general encoder on held-out behavior.

When pairs exist, fine-tune, and treat the checkpoint as a hyperparameter. Run the general sentence encoder, the domain backbone, and the general sibling under one recipe, with one learning-rate sweep, as Section A.4 did. On this protocol the domain sibling won mean reciprocal rank by 0.0031 and did not win Recall@50. Measure the domain checkpoint. Adopt it when the gain on the metric you will ship is

worth pinning and re-embedding a second model. If validation cannot tell the starting points apart, ship the cheapest. Keep co-visitation in the table. On this run it leads mean reciprocal rank, and hiding it would have made every encoder look more successful than the counts allow.

Graded labels, such as ESCI's exact, substitute, complement, and irrelevant, call for a loss that sees the grades. CoSENT is that loss, and it is how the RexBERT authors evaluated similarity (Huang et al., 2024; Bajaj and Garg, 2026; Reddy et al., 2022). If multiple-negatives ranking on binarized labels matches CoSENT on validation, the graded loss is not earning its keep. Exercise A.3 is that test. Combining the encoder with a lexical index is Appendix C.

SKUs and part numbers are a tokenizer problem first. RexBERT keeps a general vocabulary of 50,368 tokens (Bajaj and Garg, 2026). SciBERT built a domain vocabulary (Beltagy et al., 2019). Count tokens per product, and how identifiers split, before a pretraining run. If the counts do not differ, the vocabulary is not the bottleneck. Exact identifier match belongs to a lexical index, in Appendix C, and Chapter 10 works the tokenization check.

Long text is where an 8,192-token encoder can read pages a short model drops. all-MiniLM-L6-v2 truncates at 256 word pieces by default (Sentence-Transformers, 2021). This run capped every model at 128 tokens, and the median description is 24 words, so that advantage was not tested. A truncation ablation that does not move the metric retires the argument for that catalog.

Amazon-M2 spans UK, DE, JP, ES, FR, and IT (Jin et al., 2023). RexBERT-micro is tagged English, and the e-commerce corpus is filtered to English (Bajaj and Garg, 2026). Prefer a multilingual encoder, or fine-tune per locale. Exercise A.5 is the check.

Token-level tasks are where domain-adaptive pretraining has the strongest published support: classification after a task head (Gururangan et al., 2020), and the token-recovery and GLUE results the RexBERT authors report after fine-tuning (Bajaj and Garg, 2026). Retire the preference if a general backbone on the same labels matches. Section A.5 is not that test.

Serving cost follows parameter count. Seventeen million and MiniLM's roughly 23 million are similar. A 150-million or 400-million encoder costs several times more per item on a full re-embedding. Compression is Section 14.8.4, and latency targets are Section 14.2.4. The micro-scale gain above is not a reason to jump to the large checkpoint without that measurement.

Prefer a checkpoint with a paper, a license a review can read, and a pinned revision. RexBERT-micro is Apache-2.0 and Ettin-17m is MIT on the hub tags checked 13 November 2026. Research releases move. The pin is the mitigation. This row has no metric falsifier.

Three checks catch an MLM encoder used as an embedding model. A mean cosine near the 0.98 in Figure A.3 means the space has not opened. Neighbors that share a brand or a size token, and do not share a function, are a reading of the list; this protocol did not score that overlap. Retrieval well below a small contrastive model is the zero-shot block of Table A.3. Any one of the three is a reason to fine-tune before the vectors go into an index.

The same objective point applies to decoder-based embedding models, which Chapter 12 takes up. A model trained to generate the next token still needs a similarity objective before its vectors are a retrieval space. This appendix does not compare those models.

Table A.4: When a domain-specialized encoder is worth adopting. A falsifier is a measurement that would retire the row on the reader's own data.

Situation	Recommendation	Falsifier
Catalog text only	Start from a contrastively trained general encoder. Use TSDAE if the wording needs adaptation.	Zero-shot domain retrieval matches the general encoder on held-out behavior.
Behavioral pairs	Fine-tune the general encoder, the domain backbone, and its sibling under one recipe.	Starting points are indistinguishable on validation. Ship the cheapest.
Graded labels	Use a loss that respects the grades, such as CoSENT.	Multiple-negatives ranking on binarized labels matches it.
SKUs and jargon	Check tokenization first. RexBERT keeps a general vocabulary.	Token counts and identifier splits do not differ.
Long product text	Test a long-context encoder. This catalog did not stress it.	A truncation ablation does not move the metric.
Several languages	Prefer a multilingual encoder, or fine-tune per locale.	A per-locale table shows no gap.
Token tasks or reranking	Domain backbones are most defensible after task fine-tuning.	A general backbone on the same labels matches.
Tight latency	Parameter count dominates. Measure before a larger domain model.	Tail latency and re-embedding time fit the budget.
Maintenance	Prefer a paper, a readable license, and a pinned revision.	None. This is a governance rule.

A.7 Summary

The released RexBERT encoder is a masked language model. The converted RexBERT sentence encoder is that backbone after mean pooling, L2 normalization, and contrastive fine-tuning. Pooled states from the released encoder are anisotropic on this catalog: random pairs and next-item pairs both sit near cosine 0.98 until fine-tuning opens the space.

Discard the masked-token head, mean-pool, normalize, and train with in-batch negatives when pairs exist, with CoSENT when grades exist, and with TSDAE when only unlabeled text exists. Select checkpoints by retrieval on sessions held out of the pairs.

On the pre-registered UK protocol, fine-tuning raises RexBERT-micro's mean reciprocal rank by 0.037 over its zero-shot start. Against Ettin-17m, under the same recipe, the e-commerce phase adds 0.0031, supported on mean reciprocal rank, with all three seeds positive. Recall@50 does not separate the

siblings. Co-visitation, which reads no text, still leads mean reciprocal rank on the 20,000 queries, at 0.235 against 0.203, because many test pairs were already counted in training. The encoders lead Recall@50 and the unseen pairs. Fine-tune, keep the sibling and the co-visitation row, and adopt the domain checkpoint when the gain on the metric you will ship is worth a separate model.

A.8 Exercises

Companion code is `code/appendix_rexbert/`. `run_all.py` runs `cpu_small`, a teaching profile whose metrics are not Section A.5, or `gpu_full`. Durations are in the README.

Exercise A.1: See the cone, then see the tokens

Run `python run_all.py --profile cpu_small` with `--data_dir` on the Amazon-M2 files, then `python make_figures.py` on that profile's metrics. A fill-mask demonstration is written from the prompts in `diagnostics.py` and flagged as a demonstration. You are done when your cosine plots show why a blank-filling guess does not predict nearest-neighbor retrieval.

Exercise A.2: Change the pair

In `data.py`, contrastive pairs are the last previous item and the next item. Co-visitation already counts every consecutive step. Build the contrastive pairs from those steps, keep the session split, and rerun a profile. Report warm and cold Recall@50 with intervals. You are done when you can say whether the extra pairs moved the cold slice.

Exercise A.3: Match the loss to graded labels

`run_all.py` does not train on ESCI. Add CoSENT beside `_loss` in `train.py` and compare it with multiple-negatives ranking on binarized English ESCI grades (Reddy et al., 2022), same validation queries. The Shopping Queries files are not in `code/appendix_rexbert/`, and the reader adds a CoSENT loss beside `_loss`. Hybrid retrieval stays in Appendix C.

Exercise A.4: Adapt without behavioral pairs

Fit TSDAE (Wang et al., 2021) to UK titles only, from MiniLM, and compare it on the test sessions with MiniLM zero-shot and with MiniLM fine-tuned on pairs. The profiles do not include this stage. You are done when the three conditions share one test table.

Exercise A.5: Cross a locale

`prepare_dataset` takes a locale; `config.py` defaults to UK. Score UK-fine-tuned RexBERT-micro on the DE catalog against a multilingual sentence encoder, with intervals. You are done when the table shows whether that fine-tune transferred. That is the multilingual falsifier in Section A.6.

References

Bajaj, R., and Garg, A. (2026). "RexBERT: Context Specialized Bidirectional Encoders for E-commerce." arXiv:2602.04605. https://arxiv.org/abs/2602.04605

Beltagy, I., Lo, K., and Cohan, A. (2019). "SciBERT: A Pretrained Language Model for Scientific Text." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. arXiv:1903.10676. https://arxiv.org/abs/1903.10676

Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Sepahvand, N. M., Raff, E., Madan, K., Voleti, V., Ebrahimi Kahou, S., Michalski, V., Serdyuk, D., Arbel, T., Pal, C., Varoquaux, G., and Vincent, P. (2021). "Accounting for Variance in Machine Learning Benchmarks." Proceedings of Machine Learning and Systems 3. arXiv:2103.03098. https://arxiv.org/abs/2103.03098

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. Association for Computational Linguistics. https://aclanthology.org/N19-1423

Ethayarajh, K. (2019). "How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. arXiv:1909.00512. https://arxiv.org/abs/1909.00512

Gao, T., Yao, X., and Chen, D. (2021). "SimCSE: Simple Contrastive Learning of Sentence Embeddings." Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. arXiv:2104.08821. https://arxiv.org/abs/2104.08821

Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., and Smith, N. A. (2020). "Don't Stop Pretraining: Adapt Language Models to Domains and Tasks." Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics. arXiv:2004.10964. https://arxiv.org/abs/2004.10964

Henderson, M., Al-Rfou, R., Strope, B., Sung, Y., Lukács, L., Guo, R., Kumar, S., Miklos, B., and Kurzweil, R. (2017). "Efficient Natural Language Response Suggestion for Smart Reply." arXiv:1705.00652. https://arxiv.org/abs/1705.00652

Huang, X., Peng, H., Zou, D., Liu, Z., Li, J., Liu, K., Wu, J., Su, J., and Yu, P. S. (2024). "CoSENT: Consistent Sentence Embedding via Similarity Ranking." IEEE/ACM Transactions on Audio, Speech, and Language Processing 32, 2800-2813. https://doi.org/10.1109/TASLP.2024.3402087

Jin, W., Mao, H., Li, Z., Jiang, H., Luo, C., Wen, H., Han, H., Lu, H., Wang, Z., Li, R., Li, Z., Cheng, M., Goutam, R., Zhang, H., Subbian, K., Wang, S., Sun, Y., Tang, J., Yin, B., and Tang, X. (2023). "Amazon-M2: A Multilingual Multi-locale Shopping Session Dataset for Recommendation and Text Generation." Advances in Neural Information Processing Systems, Datasets and Benchmarks. arXiv:2307.09688. https://arxiv.org/abs/2307.09688

Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., and Kang, J. (2020). "BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining." Bioinformatics 36(4), 1234-1240. https://doi.org/10.1093/bioinformatics/btz682

Li, B., Zhou, H., He, J., Wang, M., Yang, Y., and Li, L. (2020). "On the Sentence Embeddings from Pre-trained Language Models." Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. arXiv:2011.05864. https://arxiv.org/abs/2011.05864

Reddy, C. K., Márquez, L., Valero, F., Rao, N., Zaragoza, H., Bandyopadhyay, S., Biswas, A., Xing, A., and Subbian, K. (2022). "Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search." arXiv:2206.06588. https://arxiv.org/abs/2206.06588

Reimers, N., and Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. arXiv:1908.10084. https://arxiv.org/abs/1908.10084

Sentence-Transformers. (2021). "all-MiniLM-L6-v2." Hugging Face model card. Accessed 13 November 2026. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

van den Oord, A., Li, Y., and Vinyals, O. (2018). "Representation Learning with Contrastive Predictive Coding." arXiv:1807.03748. https://arxiv.org/abs/1807.03748

Wang, K., Reimers, N., and Gurevych, I. (2021). "TSDAE: Using Transformer-based Sequential Denoising Auto-Encoder for Unsupervised Sentence Embedding Learning." Findings of the Association for Computational Linguistics: EMNLP 2021. arXiv:2104.06979. https://arxiv.org/abs/2104.06979

Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Cooper, N., Adams, G., Howard, J., and Poli, I. (2024). "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference." arXiv:2412.13663. https://arxiv.org/abs/2412.13663

Weller, O., Ricci, K., Marone, M., Chaffin, A., Lawrie, D., and Van Durme, B. (2025). "Seq vs Seq: An Open Suite of Paired Encoders and Decoders." arXiv:2507.11412. https://arxiv.org/abs/2507.11412